TractionTherm — Django backend design

Modules, models, tables, services and APIs for the Loco Motor Test system

This document describes the server that will replace the browser-only storage of the TractionTherm motor-monitoring web UI. It is written for the backend team: it lists the Django apps to create, every model and database table with its fields, the services that hold the business rules, the REST and WebSocket APIs the existing front end needs, the export formats, permissions, operations and a build plan.

Everything here is derived from what the current front end does today (the 13 pages and `assets/js/app.js`), so the backend can be built without changing how the app looks or behaves.

ITEM	VALUE
Product	TractionTherm · traction-motor temperature monitoring (heat-run tests)
Front end	Static HTML / CSS / JavaScript, 13 pages, no framework
Target backend	Python 3.12, Django 5.x, Django REST Framework, PostgreSQL 16
Real-time	Django Channels (WebSockets) with Redis
Background work	Celery with Redis (evaluation, minute trend, offline detection, exports)
Status	Design for implementation · version 1.46

What changed in version 1.46

AREA	CHANGE
Downloads	Every download button asks **CSV or PDF** first. Excel is no longer offered. The Results table and the Activity log now download as real PDF files (the separate Print / PDF button is gone).
Settings	The *Real-time interval* setting was removed from the Settings page. The device sampling rate (2 s) is a deployment setting (`LIVE_SECONDS`), not a user setting. Settings now has six boxes.
Real-time graphs	One *range* picker only (last 5 / 15 / 30 min, last 1 h, whole test). The interval follows the range automatically (2 s, 10 s, 30 s, 1 min, 5 min). Time labels fall on round clock times. The "Live · every 2 s" tag was removed.
Monitor page	New **Live readings** card: pick any motor being checked, see its latest values with °C/min, the live graph and the readings table.
Sensor names	Shown everywhere as the motor's name with the slot in brackets: DE (T1), NDE (T2) for three-phase, PE (T1), CE (T2) for conventional.
Readings log	Each reading is judged against the temperature limit: **Within limit / Above limit** (a single reading is never "Stable").
Dashboard	A device that stops reporting while it checks motors shows an **Offline** alert; the alert count includes it.
Results test view	Head: Edit details, Download CSV / PDF, ← Back (to Results). The change log is the last card.
Pagination	Every list table has **Rows per page** (10 / 25 / 50 / 100).
Motors page	The *Change* link next to an assigned motor's device was removed; *Assign* remains for unassigned motors.
Roles page	A legend explains "—" (action not applicable to that area) and the grey tick (always on).

Contents

What changed in version 1.46

1. What the system does

1.1 Domain in one page

1.2 Pages and what each needs from the server

1.3 The test view on Results

2. Architecture

2.1 Technology choices

2.2 Project layout

3. Modules (Django apps)

4. Models and database tables

4.1 accounts

4.2 devices

4.3 motors

4.4 testing

4.5 telemetry

4.6 conditions

4.7 activity

4.8 Relationships at a glance

5. Services (business logic)

5.1 Evaluation rules

5.2 Downloads and report formats

5.3 Results queries

5.4 Background jobs (Celery beat)

6. REST API

6.1 Auth and current user

6.2 Devices

6.3 Motors

6.4 Tests (runs)

6.5 Results and exports

6.6 Settings

6.7 Users, roles, activity, dashboard

6.8 Device ingestion (machine to machine)

6.9 URL routing

7. Live updates (WebSockets)

8. Permissions

9. Moving the front end to the API

10. Security and operations

10.1 Runtime services

10.2 Environment settings

10.3 Python packages

11. Build plan

Acceptance tests to port

1. What the system does

Traction motors are heat-run tested in a loco shed. A test device sits at the bench and reads temperatures; the web app shows them live, judges each motor's condition and keeps the results.

1.1 Domain in one page

- **Device**: a Wi-Fi data logger with 2 channels. It can test at most 2 motors at once and has 5 terminals: 2 per channel (T1, T2) and 1 shared ambient sensor (TA).
- **Motor**: identified by Motor ID and a unique Stator No. (plus model and locomotive). A motor can be tested many times over its life.
- **Motor type**: *Three-phase* (stored as `AC`; sensors DE = drive end, NDE = non-drive end) or *Conventional* (stored as `DC`; PE = pinion end, CE = commutator end). The type decides the sensor names on every page and report: **DE (T1) / NDE (T2)** or **PE (T1) / CE (T2)**. Lists that mix both types use the slot names T1 / T2. The same limits apply to both types.
- **Test (a run)**: one motor on one device channel, from Switch on until Stop. Every test has its own Test ID (e.g. TEST-0007). The stator stays the same; the Test ID changes each run.
- **Reading**: one sample of T1, T2 and TA every 2 s (`LIVE_SECONDS`). TA comes from the device's one ambient sensor, so both motors on a device have the same TA at the same moment. A per-minute trend is kept for the whole test; results and reports list one reading per *interval time* (Settings, default 15 min).
- **Vibration readings**: SKF readings typed in when a motor is stopped: velocity (mm/s) and acceleration envelope (gE) on each side; saved with the test and printed on its report.
- **Condition**: *Unstable* as soon as the hotter sensor goes above the temperature limit, or when the maximum test hours pass without settling. *Stable* once the temperatures settled (the last 5 recorded readings each changed by less than 0.5 °C on both sensors) within the maximum hours. *Processing* otherwise. A completed test is always Stable or Unstable.
- **Switch on without a motor**: an operator can start reading a free channel straight away; the run gets a temporary ID (NEW-001) and the real motor is attached later (*Add details*).

1.2 Pages and what each needs from the server

PAGE	PURPOSE	SERVER DATA USED
Sign in (index)	Username / e-mail and password	auth
Dashboard	Counts by condition; alerts (motors not Stable, and devices offline while checking motors); motor temperature chart (T1 / T2 per motor against the limit, predicted final temperature) with a table: motor, device · type, T1, T2, hottest, predicted final, test time, condition	live summary, conditions, device status
Monitor	A live widget per device: its two channels with the common TA; under each sensor its rise above ambient and its change between the last two recorded readings; recent motor activity per device. **Live readings** card below: motor picker (every motor being checked), range picker, latest values with °C/min, live graph and paged readings table	devices, live readings, activity
Device	3D view of one device, both channels side by side with the common TA; per channel a real-time graph with its table, and test history	device, runs, readings, activity
Motor	One motor: live 3D view, readings, real-time graph and comparison, test history; vibration readings asked when stopped; a message when a running test is past its maximum hours	motor, run, readings, evaluation

PAGE	PURPOSE	SERVER DATA USED
Results	Completed tests only (Stable or Unstable). Key columns: Test ID, motor, condition, started, duration, rise, device · channel, model / type. Filters, quick and custom rules, readings log, Download CSV / PDF. A test opens its own view (see 1.3)	tests, readings, reports, test changes
Devices	Register and edit devices, see their motors	devices CRUD
Motors	Register, edit, assign (unassigned motors), delete motors; Download all results	motors CRUD, assignment
Settings	Six boxes and one Save: maximum test hours, interval time, last how many readings, change less than (°C), temperature must not exceed (°C), ask for vibration on stop. Below: the Processing / Stable / Unstable rules with the current values and a line per running motor	settings
Users	Accounts, enable / disable	users CRUD
Roles & permissions	Roles and their View / Create / Update / Delete grid, with a legend	roles CRUD
Activity log	Who did what; filter by category, user, period, search; Download CSV / PDF	activity
My profile	Own details (read-only) and password change	auth/me, change password

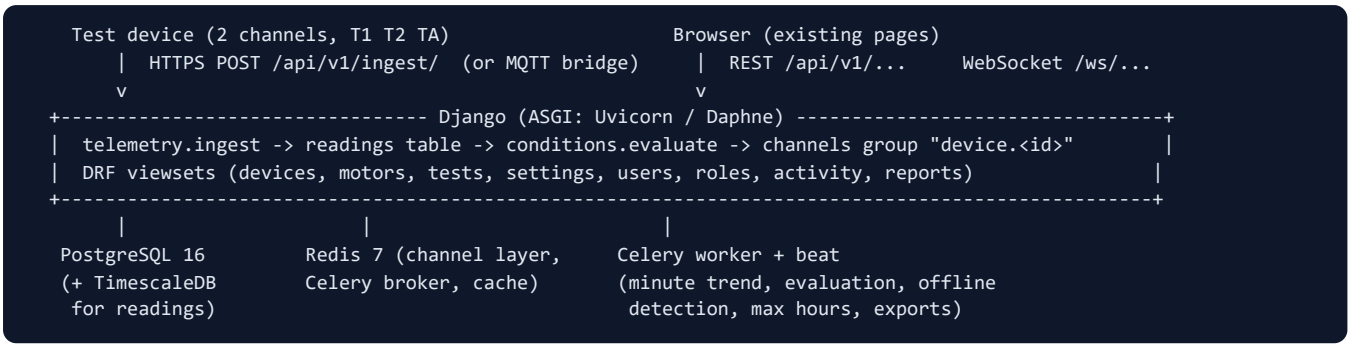
1.3 The test view on Results

Opened from a Test ID anywhere in the app (`reports.html?motor=…&test=…`). The filters are hidden. In order:

1. **Head**: Test ID · *Edit details* · *Download CSV / PDF* · ← *Back* (to Results).
2. **Full details**: every field of the test; sensor names are the test's own (e.g. "DE (T1) °C").
3. **Whole test** graph: every stored reading from the start (live while running).
4. **Real-time** graph with its paged table, latest values and °C/min (current test of the motor only).
5. The **report sheet**: readings at the interval time with DIFF, calculated results, graph.
6. Summary strip and **Result history** of the motor (every completed test), with *Download CSV / PDF*.
7. **Change log** of the test (last).

A test still in progress is not shown here: the page says "TEST-… is still in progress" with links to the motor's live page.

2. Architecture



2.1 Technology choices

CONCERN	CHOICE	WHY
Web framework	Django 5 + Django REST Framework	Admin, ORM, migrations, auth and permissions out of the box
Database	PostgreSQL 16; TimescaleDB extension for readings	A 2 s sample per channel is about 43,000 rows a day per channel; hypertables keep it fast
Live updates	Django Channels + channels-redis	Pages refresh every reading (2 s): push instead of polling
Background jobs	Celery + Redis, django-celery-beat	Minute trend, evaluation, offline detection, max-hours watch, large exports
Auth	Django session auth (browser); per-device API keys	HttpOnly session cookie for people; hashed keys for devices
API docs	drf-spectacular (OpenAPI 3)	Generated schema for the front end and device firmware
Exports	`csv` module; ReportLab for PDF	Every download is CSV or PDF (section 5.2)
Filtering	django-filter	Query-string filters on every list
Rate limits	django-axes (sign-in), DRF throttles	Brute-force protection

2.2 Project layout

```
tractiontherm/                 # Django project
  settings/  base.py  dev.py  prod.py
  asgi.py  urls.py  celery.py  routing.py
apps/
  core/          BaseModel, permission classes, pagination, error format, utils
  accounts/      users, roles, permissions, sign-in, my profile
  devices/       devices, device API keys, online status
  motors/        motors (Motor ID, Stator No., type, model, locomotive)
  testing/       test runs, Test ID sequence, switch on / start / stop / assign / claim / channel / terminals / edit
  telemetry/     readings, minute trend, ingestion, live push (consumers)
  conditions/    test settings, evaluation, heating model
  reports/       results queries, readings log, CSV and PDF exports
  activity/      audit log and per-test change log
```

Inside every app:

```
apps/<app>/
  models.py        tables (section 4)
  services.py      business rules (section 5) — views never change data directly
  selectors.py     read-only queries used by views and reports
  serializers.py   input validation and output shape
  views.py         thin DRF viewsets / APIViews
  urls.py          routes (section 6)
  permissions.py   area.action checks (section 8)
  tasks.py         Celery tasks (section 5.4)
  consumers.py     WebSocket consumers (telemetry only)
  admin.py         Django admin (read-only for activity)
  tests/           pytest tests per service and endpoint
```

3. Modules (Django apps)

APP	OWNS	DEPENDS ON
core	BaseModel (timestamps), permission classes, error format, pagination (page_size 10 / 25 / 50 / 100)	–
accounts	User, Role, RolePermission; sign-in / sign-out; my profile; permission checks	core
devices	Device, DeviceApiKey; online status; device switcher list	accounts
motors	Motor (identity and details)	accounts
testing	TestRun, TestIdSequence; Switch on, Start, Stop, Assign, Claim (Add details), Change channel, Change terminals, Edit details, Vibration	devices, motors, telemetry, conditions, activity
telemetry	Reading (2 s), TrendPoint (1 min); ingestion endpoint; WebSocket push	devices, testing
conditions	TestSettings (one set for both motor types); evaluation (settle rule, maximum hours, temperature limit); heating model	telemetry
reports	Results queries, readings log, CSV and PDF exports (test report, results table, activity log)	testing, telemetry, conditions, activity
activity	ActivityEntry (append only) with device / motor / test links	accounts

4. Models and database tables

Field types are Django field types. Table names use the `app_model` default. Every table has `created_at` and `updated_at` from `core.BaseModel` unless stated.

```
# apps/core/models.py
class BaseModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta:
        abstract = True
```

4.1 accounts

User · table `accounts_user` (custom user model, set `AUTH_USER_MODEL` before the first migration)

FIELD	TYPE	RULES / NOTES
id	BigAutoField	Primary key
username	CharField(150)	Unique, lower case; sign-in accepts username or e-mail
email	EmailField	Unique
name	CharField(120)	Display name, e.g. Arun Prakash
role	ForeignKey(Role, PROTECT)	Exactly one role per user
password	CharField	Django hash (PBKDF2 / Argon2); never stored or sent in plain text
is_active	BooleanField	Disabled accounts cannot sign in; open sessions are ended
last_login	DateTimeField(null)	Set at sign-in

Role · table `accounts_role`

FIELD	TYPE	RULES / NOTES
id	BigAutoField	Primary key
name	CharField(60)	Unique, 2–30 characters. Built in: Administrator, Supervisor, Operator, Viewer
description	CharField(200)	
is_builtin	BooleanField	Built-in roles cannot be deleted or renamed
is_admin	BooleanField	Administrator: always every permission, cannot be edited

RolePermission · table `accounts_rolepermission`

FIELD	TYPE	RULES / NOTES
role	ForeignKey(Role, CASCADE)	
key	CharField(40)	One of the keys in section 8, e.g. `motors.update`

Unique together (role, key). Granting create / update / delete also grants the area's view; removing view removes the rest of the row.

```
# apps/accounts/models.py
class Role(BaseModel):
    name = models.CharField(max_length=60, unique=True)
    description = models.CharField(max_length=200, blank=True)
    is_builtin = models.BooleanField(default=False)
    is_admin = models.BooleanField(default=False)

class RolePermission(models.Model):
    role = models.ForeignKey(Role, on_delete=models.CASCADE, related_name="permissions")
    key = models.CharField(max_length=40)
    class Meta:
        constraints = [models.UniqueConstraint(fields=["role", "key"], name="uniq_role_perm")]

class User(AbstractBaseUser, PermissionsMixin, BaseModel):
    username = models.CharField(max_length=150, unique=True)
    email = models.EmailField(unique=True)
    name = models.CharField(max_length=120)
    role = models.ForeignKey(Role, on_delete=models.PROTECT, related_name="users")
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    USERNAME_FIELD = "username"
    EMAIL_FIELD = "email"
    REQUIRED_FIELDS = ["email", "name"]
    objects = UserManager()

    def has_key(self, key: str) -> bool:
        return self.role.is_admin or self.role.permissions.filter(key=key).exists()
```

4.2 devices

Device · table `devices_device`

FIELD	TYPE	RULES / NOTES
id	CharField(16), primary key	Device ID, 3–16 letters / numbers / dashes, upper case, e.g. TMU-01
name	CharField(60)	Unique (case-insensitive), e.g. Device A
location	CharField(120)	e.g. Loco Shed · Bay 3
ip_address	GenericIPAddressField	Unique; four numbers 0–255
firmware	CharField(20)	e.g. 2.4.1
channels	PositiveSmallIntegerField	Default 2 (CH 1, CH 2)
signal	PositiveSmallIntegerField(null)	Wi-Fi bars 0–4, last reported
last_seen	DateTimeField(null)	Updated by every ingest and heartbeat
is_online	BooleanField	Derived: seen within max(10 s, 3 × LIVE_SECONDS); kept as a column so lists and alerts are cheap; set by ingest and by `detect_offline_devices`

DeviceApiKey · table `devices_deviceapikey`

FIELD	TYPE	RULES / NOTES
device	ForeignKey(Device, CASCADE)	
key_hash	CharField(128)	Hash of the key the device sends as `Authorization: Device <key>`
label	CharField(60)	
revoked_at	DateTimeField(null)	Revoked keys are refused

4.3 motors

Motor · table `motors_motor`

FIELD	TYPE	RULES / NOTES
id	BigAutoField	Primary key (internal)
motor_id	CharField(40)	Unique, upper case, e.g. TM-4471. Can be renamed; history follows the row
stator_no	CharField(40, null)	Unique when set, upper case. Required except for a temporary motor
model	CharField(120)	e.g. 6FRA 6068 · 3-phase AC
locomotive	CharField(60)	e.g. WAG-9 #31245
motor_type	CharField(2), choices AC / DC	Required. Decides sensor names (DE / NDE or PE / CE)
is_temporary	BooleanField	True for NEW-001 style motors created by Switch on
deleted_at	DateTimeField(null)	Soft delete: finished tests keep their motor

Where a motor is now is not stored here: it is the device of its open TestRun, if any. Deleting a motor is refused while it has a running test.

4.4 testing

TestRun · table `testing_testrun`

FIELD	TYPE	RULES / NOTES
id	BigAutoField	Primary key
test_id	CharField(24)	Unique, upper case, 3–24 letters / digits / dashes, e.g. TEST-0007
motor	ForeignKey(Motor, PROTECT)	
device	ForeignKey(Device, PROTECT)	Device the test ran on
channel	PositiveSmallIntegerField	1 or 2
state	CharField(10)	running / stopped / archived
started_at	DateTimeField	
stopped_at	DateTimeField(null)	Set by Stop
motor_type	CharField(2)	Copied from the motor at start, so an old test keeps its sensor names
t1_measures, t2_measures	CharField(40)	What T1 / T2 measure in this test, e.g. Drive end (DE)
result	CharField(8, null)	ok / crit when completed (shown Stable / Unstable)
reason	CharField(200)	e.g. "Temperature above the limit", "Not settled in time"
max_t1, max_t2, max_rise	DecimalField(5,1)	Peaks for fast lists and summaries
started_by, stopped_by	ForeignKey(User, SET_NULL, null)	
vib_s1_mms, vib_s1_ge	DecimalField(6,3, null)	Side 1 (DE or PE): velocity 0–100 mm/s, envelope 0–50 gE; both or none

FIELD	TYPE	RULES / NOTES
vib_s2_mms, vib_s2_ge	DecimalField(6,3, null)	Side 2 (NDE or CE)
vib_by, vib_at	ForeignKey(User, SET_NULL, null), DateTimeField(null)	Who entered them and when; only while stopped

Constraints:

- One running test per channel: partial unique index on (device, channel) WHERE state = 'running'.
- One open test per motor: partial unique index on (motor) WHERE state IN ('running', 'stopped').
- A stopped test is the motor's *current* test until a new test starts; then it becomes archived.
- A Test ID once used can never be used again.

```
# apps/testing/models.py
class TestRun(BaseModel):
    class State(models.TextChoices):
        RUNNING = "running"; STOPPED = "stopped"; ARCHIVED = "archived"
    test_id = models.CharField(max_length=24, unique=True)
    motor = models.ForeignKey("motors.Motor", on_delete=models.PROTECT, related_name="tests")
    device = models.ForeignKey("devices.Device", on_delete=models.PROTECT, related_name="tests")
    channel = models.PositiveSmallIntegerField()
    state = models.CharField(max_length=10, choices=State.choices, default=State.RUNNING)
    started_at = models.DateTimeField()
    stopped_at = models.DateTimeField(null=True, blank=True)
    motor_type = models.CharField(max_length=2, choices=[("AC", "Three-phase"), ("DC", "Conventional")])
    t1_measures = models.CharField(max_length=40)
    t2_measures = models.CharField(max_length=40)
    result = models.CharField(max_length=8, null=True, blank=True)
    reason = models.CharField(max_length=200, blank=True)
    max_t1 = models.DecimalField(max_digits=5, decimal_places=1, null=True)
    max_t2 = models.DecimalField(max_digits=5, decimal_places=1, null=True)
    max_rise = models.DecimalField(max_digits=5, decimal_places=1, null=True)
    vib_s1_mms = models.DecimalField(max_digits=6, decimal_places=3, null=True)
    vib_s1_ge = models.DecimalField(max_digits=6, decimal_places=3, null=True)
    vib_s2_mms = models.DecimalField(max_digits=6, decimal_places=3, null=True)
    vib_s2_ge = models.DecimalField(max_digits=6, decimal_places=3, null=True)
    started_by = models.ForeignKey("accounts.User", null=True, on_delete=models.SET_NULL, related_name="+")
    stopped_by = models.ForeignKey("accounts.User", null=True, on_delete=models.SET_NULL, related_name="+")
    vib_by = models.ForeignKey("accounts.User", null=True, on_delete=models.SET_NULL, related_name="+")
    vib_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["device", "channel"], condition=models.Q(state="running"),
                                    name="one_running_test_per_channel"),
            models.UniqueConstraint(fields=["motor"], condition=models.Q(state__in=["running", "stopped"]),
                                    name="one_open_test_per_motor"),
        ]
        indexes = [models.Index(fields=["state", "started_at"])]
```

TestIdSequence · table `testing_testidsequence` — single row, `last_number` PositiveIntegerField. Next free ID is `TEST-` + (last_number + 1) zero-padded to 4. Read and increment inside `select_for_update()`. Custom Test IDs typed by users only move the sequence when they follow TEST-NNNN.

4.5 telemetry

Reading · table `telemetry_reading` (TimescaleDB hypertable on `time`; no `updated_at`)

FIELD	TYPE	RULES / NOTES
test_run	ForeignKey(TestRun, CASCADE)	Indexed together with time

FIELD	TYPE	RULES / NOTES
time	DateTimeField	Device time, UTC
t1, t2, ta	DecimalField(5,1)	°C, one decimal. `ta` is the device's one ambient sensor: same value for both channels in a sample

Written every 2 s per running channel; index (test_run, time). Raw readings kept 30 days.

TrendPoint · table `telemetry_trendpoint`

FIELD	TYPE	RULES / NOTES
test_run	ForeignKey(TestRun, CASCADE)	
minute	DateTimeField	Start of the minute; unique together (test_run, minute)
t1, t2, ta	DecimalField(5,1)	Reading at that minute (whole-test graph, reports)

Kept for the life of the test.

4.6 conditions

TestSettings · table `conditions_testsettings` — single row, edited on the Settings page with one Save.

FIELD	TYPE	RULES / NOTES
max_test_hours	DecimalField(4,1)	Default 4; 0.5–48 in half hours. Stable must be reached within this time; a test may run on past it (pages show a message)
interval_minutes	PositiveSmallIntegerField	Default 15; 1–240 ("Interval time"): one reading per this many minutes from the test start in results, reports and the settle rule
settle_readings	PositiveSmallIntegerField	Default 5; 2–20 ("Last how many readings")
settle_diff	DecimalField(4,1)	Default 0.5 °C; 0.1–10 ("Change less than") on both motor sensors
max_temp	DecimalField(5,1)	Default 100 °C; 20–250 ("Temperature must not exceed"): the hotter of T1 / T2 above it makes the test Unstable at once
ask_vibration_on_stop	BooleanField	Default True
updated_by	ForeignKey(User, SET_NULL, null)	

Rejected (400) when settle_readings × interval_minutes > max_test_hours × 60 (no test could become Stable). The sampling rate is **not** a user setting any more: `LIVE_SECONDS` (default 2) is an environment setting and is sent to devices in the heartbeat reply.

Sensor names are not stored: they come from the motor type.

4.7 activity

ActivityEntry · table `activity_activityentry` (append only; no `updated_at`)

FIELD	TYPE	RULES / NOTES
id	BigAutoField	
time	DateTimeField	Server time, indexed
category	CharField(10)	auth, device, motor, test, limits, user, role, report, access
text	TextField	Human sentence, e.g. "Stopped monitoring TM-4471 on Device A CH 1 · test TEST-0001"

FIELD	TYPE	RULES / NOTES
user	ForeignKey(User, SET_NULL, null)	Plus copies of user name, username and role at the time
user_name, username, role_name	CharField	Copies, so the log stays readable after a user is deleted
devices	ManyToManyField(Device)	Devices the event belongs to (both devices for a move)
motor, test_run	ForeignKey(null, SET_NULL)	Links for "open the motor" and the test's change log

No update or delete through the API or admin.

4.8 Relationships at a glance

```
Role 1 --- * User                 Device 1 --- * TestRun * --- 1 Motor
Role 1 --- * RolePermission       TestRun 1 --- * Reading
                                  TestRun 1 --- * TrendPoint
ActivityEntry * --- * Device      ActivityEntry * --- 0..1 Motor / TestRun / User
TestSettings (1 row)              TestIdSequence (1 row)
```

5. Services (business logic)

Views stay thin: they validate input with serializers and call a service. Every service that changes data runs in `transaction.atomic()`, checks the permission, writes an ActivityEntry and publishes a WebSocket event after commit (`transaction.on_commit`).

```
# apps/testing/services.py (shape of every service)
class MonitoringService:
    @staticmethod
    @transaction.atomic
    def stop(test_run: TestRun, user: User) -> TestRun:
        require(user, "monitoring.update")
        if test_run.state != TestRun.State.RUNNING:
            raise Conflict("This test is not running.")
        test_run.state, test_run.stopped_at, test_run.stopped_by = "stopped", timezone.now(), user
        result = EvaluationService.evaluate(test_run)
        test_run.result, test_run.reason = result.final, result.reason
        test_run.save()
        ActivityService.log("test", f"Stopped monitoring {test_run.motor.motor_id} on {test_run.device.name} "
                            f"CH {test_run.channel} · test {test_run.test_id} · last result {result.label}",
                            user, devices=[test_run.device], motor=test_run.motor, test_run=test_run)
        transaction.on_commit(lambda: push("test.changed", test_run))
        return test_run
```

SERVICE	FUNCTION	RULES IT ENFORCES
TestIdService	`next_id()`, `reserve(test_id)`	Format and uniqueness against all current and archived tests; sequence under row lock
MonitoringService	`switch_on(device, motor_type)`	Device online; a free channel; creates temporary motor NEW-NNN of that type and a running test with the next Test ID
	`start(motor, device, test_id)`	Free channel on that device; archives the motor's previous stopped test; copies sensor *measures* defaults
	`stop(test_run)`	Sets stopped_at, computes final result and peaks; channel becomes free; reply says whether to ask for vibration
VibrationService	`record(test_run, side1, side2)`	Only on a stopped test; each side both values or none; at least one side; 0–100 mm/s, 0–50 gE, 3 decimals
AssignmentService	`assign(motor, device, test_id)`	Unassigned motor onto a device's free channel; starts a test. (The UI no longer offers moving an assigned motor)
	`claim(temp_test, motor, test_id)`	*Add details* on a switched-on run: the chosen or new motor takes over the run and its readings; the temporary motor is removed
ChannelService	`change(test_run, channel)`	Same device. Other channel free: move; busy: swap the two running tests. Readings stay with each test
TerminalService	`change(test_run, t1, t2, swap)`	Values from the motor type's measuring points; not both the same; `swap` (running only) exchanges t1 / t2 in every Reading and TrendPoint row and the peaks
TestEditService	`edit(test_run, ...)`	*Edit details*: terminals always; swap only while running; vibration on a stopped test; motor details (apply to all the motor's tests). Test ID, device and channel are read-only
IngestService	`ingest(device, samples)`	Authenticates the device key; maps channel to its running test; stores readings; updates last_seen / is_online; triggers push. Heartbeat reply carries `live_seconds`

SERVICE	FUNCTION	RULES IT ENFORCES
EvaluationService	`evaluate(test_run)`	Section 5.1; returns the structure the UI shows (checks, condition, reason, heating model)
ReportService	`test_csv(tests)`, `test_pdf(tests)`, `table_csv(rows)`, `table_pdf(rows)`	Section 5.2. Every download is CSV or PDF
ResultsService	`completed(filters, rules)`, `readings(test_run, range)`	Completed tests only (Stable / Unstable; a saved Processing counts as Unstable). Real-time readings: range → interval (section 6.4)
ReadingsLogService	`readings(filters, range, every)`	One row per reading at the interval; *Limit check* = Within limit / Above limit
ChangeLogService	`changes(test_run)`	ActivityEntry rows linked to the test (earlier Test IDs included), newest first
ActivityService	`log(category, text, user, devices, motor, test_run)`, `export(filters, fmt)`	Immutable; CSV or PDF export of the filtered log
AuthService	`sign_in()`, `sign_out()`, `change_password(user, current, new)`	Username or e-mail; disabled accounts refused; failures logged and rate limited; password ≥ 6 characters and different; other sessions ended
DeviceService	`create`, `update`, `delete`	ID 3–16 letters / numbers / dashes, unique; name unique; IP valid and unique; delete refused (409) while a test runs on it (message names the motors); otherwise its motors become unassigned
MotorService	`register(data, device=None, test_id=None)`, `update`, `delete`	Motor ID and Stator No. unique (message names the motor that has it); type AC / DC; register on a device starts a test; rename keeps all tests; delete refused while running
SettingsService	`get()`, `update(values, user)`	All values checked together; one activity entry listing every change; broadcasts `settings.changed`
UserService	`create`, `update`, `set_active`, `delete`	Name, username, e-mail required and unique; e-mail valid; password ≥ 6; nobody deletes or disables themselves; the last active administrator can't be removed or demoted; disabling ends sessions
RoleService	`create(name, desc, from_role)`, `update`, `set_permissions(role, keys)`, `delete`	Name 2–30, unique; built-in roles never deleted or renamed; Administrator fixed; a role with users can't be deleted; create / update / delete imply view
DashboardService	`summary()`	Counts by condition, alerts (motors not Stable + offline devices with running tests), chart rows hottest first

5.1 Evaluation rules

CHECK	VALUE	EFFECT
Motor temperature	hot = max(T1, T2); peak = highest hot so far	peak > max_temp: **Unstable** at once, also while running
Temperatures settled	last `settle_readings` recorded readings (interval apart, only up to start + max_test_hours): each |reading − previous| on T1 and T2	All < settle_diff: **Stable**; otherwise "Not settled yet" = **Processing**
Test time	running time against max_test_hours	Reached, or stopped, without settling: **Unstable** ("Not settled in time")

A test may keep running past its maximum hours; the pages say "Maximum test hours exceeded (N hours) · result … · stop the test when ready". A completed test is never Processing. A device offline shows **Offline** on live pages and an alert on the Dashboard. Rise above ambient, T1 / T2 difference and the heating model's predicted final temperature are shown as information only.

Readings log: one reading is **Above limit** when its hotter motor sensor is above max_temp, otherwise **Within limit**. The Results condition filter shows Stable / Unstable for test results and Within limit / Above limit for the readings log.

Heating model: for each of T1 and T2 the rise above ambient over test time *t* (minutes) is fitted to rise(t) = A − (A − r0)·e^(−t/τ) by least squares (grid search on τ from 1 to 600 min, closed form for A and r0). Confident when $R^2 \geq 0.8$ and the test has run at least 0.4·τ. Predicted final temperature = TA + A. Run after each minute trend point and cache per test in Redis.

5.2 Downloads and report formats

Every download button asks "CSV or PDF?" first. Excel is not offered. Buttons and what they produce:

BUTTON (PAGE)	CSV	PDF
Download CSV / PDF (Results)	Results table, every column, with the filters as header lines	Results table: key columns + Reason, every row, filters and test settings at the top
Download CSV / PDF (Results, readings view)	Readings log, every column	Readings log table
All motor results (Results) / Download all results (Motors)	Latest completed test of every motor, one block each	One report per test (see below)
Download (each test row, test history, test view head)	That test's report as CSV	That test's report
Download test results (Motor page)	The motor's current test	The motor's current test
Download CSV / PDF (Result history, test view)	The motor's result history table	Result history table
Download CSV / PDF (Activity log)	Filtered log: Time, User, Username, Role, Category, Activity	Filtered log as a table

Before a test report is downloaded, each stopped test without vibration readings asks for them (Save and continue, Skip, Skip all, Cancel; only for users with monitoring.update; never for a running test).

Test report — PDF (A4 landscape): for each test a *Full details* page (header band "FULL DETAILS · test ID · TM NO"; every field as label / value cells, 4 a row, the reason in the last cell; "Generated … · readings every N min"), then the *sheet* page(s): header band TM NO / DATE; readings table TIME · sensor 1 · DIFF · sensor 2 · DIFF (24 rows a page; DIFF green when |DIFF| < settle_diff); calculated results; graph of both sensors (title TM NO, x axis "Time (At 15min)", y axis "Temperature").

Test report — CSV: per test, blocks separated by blank lines:

```
Report,TractionTherm test results
Generated,"Nov 10, 2026, 12:42 PM"

Test ID,TEST-0005
TM NO,TM-5121
Date,10-11-2026
Device,Device B · CH 1
Result,Unstable

Details
<label>,<value>            (every Full details field, sensor names of the test, e.g. "DE (T1) °C")

Calculated results,DE (T1),NDE (T2)
Runtest duration,1:00,1:00
...

Readings (every 15 min)
Time,DE (T1) °C,DE (T1) diff,NDE (T2) °C,NDE (T2) diff,TA °C
"Nov 10, 2026, 11:32 AM",31.8,0.0,31.7,0.0,31.1
```

Calculated results (same numbers on screen, CSV and PDF):

PART	FORMULA
Runtest duration	last row time − first row time (H:MM)
Initial / final ambient	TA of the first / last row
Final temp	each sensor on the last row
Temp raise	final temp − final ambient, per sensor
Max diff in last 1 hr	highest − lowest reading in the last 60 min of rows, per sensor
SKF reading (each side)	velocity mm/s and envelope gE, only when entered

Table PDF (Results table, readings log, result history, activity log): A4 landscape; title band with the generated time; filters line; column headings repeated on every page; long text wraps inside its column; condition coloured (Stable / Within limit green, Unstable / Above limit red); footer "<title> · page N of M". PDF fonts are WinAnsi: → is written as ->, Δ as "diff", τ as "tau".

CSV files are UTF-8 with a BOM (opens correctly in Excel), comma separated, quoted when needed. Exports are never paged.

5.3 Results queries

- Completed tests only: state stopped or archived, result Stable or Unstable.
- Filters: search (all fields or one: test, motor, stator, model, locomotive, device, location, reason, reading time), results type (test results / readings), device, motor, condition, condition type (temperature above the limit, not settled in time — test results only), time range and reading interval (readings only).
- Custom rules: field · operator · value rows, *match all* or *match any*; quick rules from the limits (hottest probe above max_temp, Unstable only, not settled in time, not settled yet, monitored over 1 h).

5.4 Background jobs (Celery beat)

JOB	WHEN	WHAT IT DOES
roll_minute_trend	every minute	For every running test, write the minute TrendPoint from that minute's raw readings
evaluate_running_tests	after each trend point	EvaluationService for every running test; cache in Redis; push `evaluation`

JOB	WHEN	WHAT IT DOES
watch_max_hours	every minute	A running test past max_test_hours gets its final condition, an activity entry and a `test.changed` push with the "maximum test hours exceeded" notice; it keeps running until stopped
detect_offline_devices	every 10 s	Device not seen within max(10 s, 3 × LIVE_SECONDS): `is_online = False`, push `device.status`; Dashboard alert and e-mail if a test was running on it
purge_raw_readings	nightly 02:00	Delete raw Reading rows older than 30 days (trend, tests and activity are kept)
build_export	on request	Large CSV / PDF exports (all motors, many tests) built in the worker, offered as a download link
yearly_archive	1 January	Move the previous year's tests, trend and activity to archive tables (still readable on Results)

Every job is idempotent and logs failures.

6. REST API

Base path `/api/v1/`. JSON, UTC ISO-8601 times, temperatures and rates in °C with one decimal. Errors:

```
{"error": {"code": "channel_busy", "message": "Both channels on Device A are in use — switch one off first.",
"field": null}}
```

with 400 (validation), 401 (not signed in), 403 (no permission), 404, 409 (conflict: busy channel, used Test ID, running test).

Pagination on every list: `page`, `page_size` (10, 25, 50, 100; max 200). Response `{count, next, previous, results}`.

Downloads: every export endpoint takes `format=csv|pdf` and returns the file with `Content-Disposition: attachment; filename="tractiontherm-<what>-<yyyymmdd-hhmm>.<ext>"`. Large exports return 202 with a job id; `GET /exports/{job}/` gives the link when ready.

6.1 Auth and current user

METHOD	PATH	PERMISSION	DOES
POST	/auth/sign-in/	public	Body: login (username or e-mail), password. Sets the session cookie; returns the user with role and permission keys
POST	/auth/sign-out/	signed in	Ends the session
GET	/auth/me/	signed in	Current user, role, permission keys (the UI hides buttons with these); feeds My profile
POST	/auth/me/password/	signed in	Body: current_password, new_password. 400 with a message if wrong / invalid; 204 on success; rate limited

6.2 Devices

METHOD	PATH	PERMISSION	DOES
GET	/devices/?search=&status=online|offline	devices.view or monitor.view	List with online state, channels in use, running motors
POST	/devices/	devices.create	Create (id, name, location, ip). 400 messages: missing field; bad ID; ID / name / IP used by another device; bad IP
GET	/devices/{id}/	devices.view	One device with both channels and its stopped motors
PATCH	/devices/{id}/	devices.update	Edit name, location, IP (same checks); ID read-only
DELETE	/devices/{id}/	devices.delete	409 while a test runs on it (names the motors); else motors become unassigned
POST	/devices/{id}/switch-on/	monitoring.create	Body: motor_type (AC / DC). Starts the free channel with a temporary motor; 409 when both channels are busy
GET	/devices/{id}/activity/?limit=5	activity.view	Recent motor and test events of this device

6.3 Motors

METHOD	PATH	PERMISSION	DOES
GET	/motors/?device=&condition=&search=&unassigned=1	motors.view	List with device, current test, live condition
POST	/motors/	motors.create	Register (motor_id, stator_no, motor_type, model, locomotive); optional device + test_id to start at once
GET	/motors/{id}/	monitor.view	Details, current test, live readings, evaluation
PATCH	/motors/{id}/	motors.update	Edit details (all its tests)
DELETE	/motors/{id}/	motors.delete	Refused while running
POST	/motors/{id}/assign/	motors.update	Body: device, test_id. Unassigned motor onto the device's free channel
GET	/motors/{id}/tests/	monitor.view	Test history, newest first
GET	/motors/{id}/report/?format=csv|pdf	reports.create	The motor's current test report (Motor page *Download test results*)

6.4 Tests (runs)

METHOD	PATH	PERMISSION	DOES
GET	/tests/?motor=&device=&condition=&ctype=&search=&rules=	reports.view	Results table: completed tests only; condition ok / crit
GET	/tests/{test_id}/	reports.view	One test: Full details, summary, peaks, terminals, result, reason
PATCH	/tests/{test_id}/	motors.update	*Edit details*: t1/t2 measures; swap (running only, 409 after); motor details; vibration on a stopped test (also needs monitoring.update). test_id, device, channel read-only (400 if sent)
POST	/tests/{test_id}/start/	monitoring.update	Switch a stopped motor on again (new test, next Test ID)
POST	/tests/{test_id}/stop/	monitoring.update	Stop. Reply includes `ask_vibration`
PUT	/tests/{test_id}/vibration/	monitoring.update	side1 / side2 {mms, ge} or null; 409 while running
POST	/tests/{test_id}/claim/	motors.update	*Add details* on a switched-on run: existing motor or new motor fields, test_id
POST	/tests/{test_id}/channel/	monitoring.update	Body: channel. Moves or swaps
POST	/tests/{test_id}/terminals/	monitoring.update	t1_measures, t2_measures, swap

METHOD	PATH	PERMISSION	DOES
GET	/tests/{test_id}/readings/?range=5|15|30|60|all	reports.view	Real-time / whole-test points. The interval follows the range: 5 → every 2 s, 15 → 10 s, 30 → 30 s, 60 → 1 min, all → 5 min (never finer than LIVE_SECONDS); one reading per interval counted back from the newest; raw samples for the last hour, minute trend before it
GET	/tests/{test_id}/sheet/	reports.view	Report sheet rows at the interval time (TIME, sensors, DIFF, TA) and calculated results
GET	/tests/{test_id}/evaluation/	monitor.view	Checks, condition, heating model
GET	/tests/{test_id}/report/?format=csv|pdf	reports.create	This test's report (section 5.2)
GET	/tests/{test_id}/changes/	reports.view	Change log: time, user name, username, text; newest first
GET	/test-ids/next/	monitoring.create	Next free Test ID to pre-fill forms

6.5 Results and exports

METHOD	PATH	PERMISSION	DOES
GET	/reports/readings/?range=60&every=15&device=&motor=&cond=&search=	reports.view	Readings log rows with *Limit check*
POST	/reports/export/	reports.create	Body: type (summary / readings), filters, rules, format (csv / pdf). The Results table download
POST	/reports/tests/	reports.create	Body: test_ids[], format. Several test reports in one file
GET	/reports/all-motors/?format=csv|pdf	reports.create	Latest completed test of every motor, one report each

6.6 Settings

METHOD	PATH	PERMISSION	DOES
GET	/settings/	limits.view	max_test_hours, interval_minutes, settle_readings, settle_diff, max_temp, ask_vibration_on_stop, plus read-only live_seconds
PUT	/settings/	limits.update	All six values validated together (400 if the settle rule can't fit in the maximum hours). One activity entry; broadcasts `settings.changed`

6.7 Users, roles, activity, dashboard

METHOD	PATH	PERMISSION	DOES
GET / POST	/users/?role=&search=	users.view / users.create	List, create
PATCH / DELETE	/users/{id}/	users.update / users.delete	Edit, enable / disable, delete (not yourself, not the last administrator)

METHOD	PATH	PERMISSION	DOES
GET / POST	/roles/	roles.view / roles.create	List with permission keys and user counts; create (optionally copying another role)
PATCH / DELETE	/roles/{id}/	roles.update / roles.delete	Rename / describe custom roles; set permissions; built-in roles not deleted or renamed; Administrator fixed
GET	/permissions/	roles.view	Permission catalogue: area, action, label, description; actions an area doesn't have are absent (the UI shows "—")
GET	/activity/?category=&user=&period=&search=	activity.view	Paged, newest first
GET	/activity/export/?format=csv|pdf&...	activity.view	Same filters
GET	/dashboard/	monitor.view	Counts, alerts (motors and offline devices), chart / table rows

6.8 Device ingestion (machine to machine)

METHOD	PATH	PERMISSION	DOES
POST	/ingest/	device API key	Samples; 204 on success; 409 if the channel has no running test (samples dropped and reported)
POST	/ingest/heartbeat/	device API key	ip, firmware, signal; keeps the device online when idle; reply `{"live_seconds": 2}`

```
POST /api/v1/ingest/              Authorization: Device <api-key>
{
  "device": "TMU-01",
  "samples": [
    {"time": "2026-11-02T06:31:04Z", "channel": 1, "t1": 50.2, "t2": 53.1, "ta": 31.0},
    {"time": "2026-11-02T06:31:04Z", "channel": 2, "t1": 57.0, "t2": 59.0, "ta": 31.0}
  ]
}
```

Devices may batch several seconds of samples. The same TA is sent with both channels.

6.9 URL routing

```
# tractiontherm/urls.py
urlpatterns = [
    path("api/v1/auth/", include("apps.accounts.urls")),
    path("api/v1/", include("apps.devices.urls")),
    path("api/v1/", include("apps.motors.urls")),
    path("api/v1/", include("apps.testing.urls")),
    path("api/v1/", include("apps.telemetry.urls")),      # /ingest/
    path("api/v1/", include("apps.conditions.urls")),     # /settings/
    path("api/v1/", include("apps.reports.urls")),        # /reports/, /dashboard/
    path("api/v1/", include("apps.activity.urls")),
    path("api/schema/", SpectacularAPIView.as_view()),
    path("admin/", admin.site.urls),
]
# tractiontherm/routing.py
websocket_urlpatterns = [
    path("ws/live/", LiveConsumer.as_asgi()),
    path("ws/devices/<str:device_id>/", DeviceConsumer.as_asgi()),
    path("ws/tests/<str:test_id>/", TestConsumer.as_asgi()),
    path("ws/activity/", ActivityConsumer.as_asgi()),
]
```

7. Live updates (WebSockets)

SOCKET	JOINED BY	MESSAGES PUSHED
/ws/live/	Monitor (widgets and Live readings card), Dashboard	`device.status` (online, signal), `reading` (device, channel, test_id, t1, t2, ta, condition) every 2 s, `test.changed`, `settings.changed`, `alert` (offline device)
/ws/devices/{id}/	Device page	readings for both channels, `test.changed`, `activity.new`
/ws/tests/{test_id}/	Motor page, Results test view	`reading`, `evaluation` (every minute), `test.changed`
/ws/activity/	Monitor (per-device lists)	`activity.new` with its device ids

Sockets use the same session as the REST API and check monitor.view (activity.view for activity). Groups: `device.<id>`, `test.<test_id>`, `activity`, `live`.

8. Permissions

Keys are `area.action`. The server checks them on every request (a DRF permission class reads the key from the view); the front end only hides buttons. Administrator always has all keys.

KEY	ALLOWS	ADMIN	SUPER.	OPER.	VIEWER
monitor.view	Live readings, dashboard, motor pages (always on)	Yes	Yes	Yes	Yes
monitoring.create	Switch on a free channel	Yes	Yes	Yes	–
monitoring.update	Start / stop, change channel, change terminals, vibration	Yes	Yes	Yes	–
motors.view / create / update / delete	Motors page, register, edit (incl. Edit details, assign), delete	Yes	Yes	view	view
devices.view / create / update / delete	Devices and device pages, add, edit, delete	Yes	Yes	view	view
limits.view / update	Settings page, change the test settings	Yes	Yes	view	view
reports.view / create	Results page, downloads (CSV / PDF)	Yes	Yes	Yes	view
activity.view	Activity log and recent-activity lists	Yes	Yes	–	–
users.view / create / update / delete	Users	Yes	–	–	–
roles.view / create / update / delete	Roles & permissions	Yes	–	–	–

Actions an area does not have are not keys at all (e.g. no `reports.delete`, no `activity.create`); the Roles page shows them as "—". My profile needs no key.

```
# apps/core/permissions.py
class HasKey(BasePermission):
    def has_permission(self, request, view):
        key = getattr(view, "permission_keys", {}).get(request.method)
        return key is None or (request.user.is_authenticated and request.user.has_key(key))
```

9. Moving the front end to the API

Today every page keeps its data in the browser's localStorage. Each key maps to an endpoint; replace the `store.get` / `store.set` calls in app.js with fetch calls and the simulated `tick()` with the WebSocket.

BROWSER KEY TODAY	HOLDS	REPLACE WITH
tt-devices-v6	Devices, their motors, current tests, test history	/devices/, /motors/, /tests/
tt-spare-v1	Motors not on a device	/motors/?unassigned=1
tt-test-seq	Last Test ID number	/test-ids/next/
tt-limits-v5, tt-recording-v1, tt-vibration-v1	Test settings	/settings/
tt-live-v1	Old sampling-rate setting (no longer editable)	drop; LIVE_SECONDS from /settings/ (read-only)
tt-users-v2, tt-roles-v1, tt-grants-v3	Users (plain passwords), roles, permissions	/users/, /roles/, /auth/me/
tt-activity-v1	Activity log (max 2000)	/activity/
tt-rt-pick	Real-time range choice	keep in the browser
tt-theme	Light / dark	keep in the browser
tt-session (sessionStorage)	Signed-in user id	Server session cookie + /auth/me/
tick() every 2 s	Simulated readings	/ws/live/ and /ws/tests/{id}/
Blob downloads built in app.js	CSV / PDF made in the browser	export endpoints (format=csv|pdf)

The front end keeps all its screens; only its data layer changes, and evaluation, heating model and exports move to the server so every user, report and export sees the same numbers.

10. Security and operations

- **Passwords**: hashed by Django; users change their own on My profile; reset by an administrator or e-mail link. The demo accounts must not be carried over.
- **Sessions**: HttpOnly, Secure, SameSite=Lax cookie; CSRF on unsafe requests; sign-out and disabling end sessions.
- **Brute force**: django-axes, 5 failures per 15 min; failures logged to the activity log.
- **Devices**: one API key per device, stored hashed, HTTPS only; revoke from the admin.
- **Audit**: ActivityEntry append only; admin read-only.
- **Time**: store UTC; show Asia/Kolkata.
- **Retention**: raw readings 30 days; minute trend, tests and activity kept (yearly archive).
- **Backups**: nightly PostgreSQL dump plus point-in-time recovery; test a restore monthly.
- **Deployment**: Docker images for web (ASGI), Celery worker, Celery beat; Nginx serving the static pages and proxying /api and /ws.
- **Monitoring**: alert when a device stops sending while a test runs, and on ingestion errors.

10.1 Runtime services

SERVICE	SOFTWARE	DOES	NOTES
web	Django 5 on Uvicorn / Daphne	REST /api/v1, WebSockets /ws, ingestion, admin	2+ processes behind Nginx; stateless
worker	Celery worker	Evaluation, jobs (5.4), exports, alerts	Same image as web
beat	Celery beat	Starts the scheduled jobs	Exactly one instance
redis	Redis 7	Channel layer, Celery broker, evaluation cache	Private network only
db	PostgreSQL 16 (+ TimescaleDB)	All data	Nightly dump + PITR
nginx	Nginx	HTTPS, static pages, proxy /api and /ws	HTTP → HTTPS; WebSocket upgrade headers

10.2 Environment settings

VARIABLE	EXAMPLE	USED FOR
DJANGO_SECRET_KEY	(long random)	Sessions and CSRF; never in the code
DATABASE_URL	postgres://tt:… @db:5432/tractiontherm	PostgreSQL
REDIS_URL	redis://redis:6379/0	Channels, Celery, cache
ALLOWED_HOSTS / CSRF_TRUSTED_ORIGINS	tt.railshed.in / https://tt.railshed.in	Host name
TIME_ZONE	Asia/Kolkata	Times on pages and reports
LIVE_SECONDS	2	Device sampling rate (sent in heartbeat replies)
EMAIL_HOST / EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	smtp.railshed.in	Password resets and alerts
ALERT_EMAILS	shed-supervisor@railshed.in	Device-offline and ingestion alerts

VARIABLE	EXAMPLE	USED FOR
DEBUG	False	Always False in production

10.3 Python packages

```
Django>=5.0  djangorestframework  django-filter  drf-spectacular  channels  channels-redis
daphne (or uvicorn[standard])  celery  django-celery-beat  redis  psycopg[binary]
reportlab  django-axes  argon2-cffi  pytest-django  factory-boy
```

11. Build plan

PHASE	SCOPE	DONE WHEN
1. Foundation	Project, core, accounts: custom user, roles, permissions, sign-in, /auth/me/, change password, admin	Users sign in on the server; the UI hides buttons from /auth/me/
2. Registry	Devices, motors, settings endpoints and validation	Devices, Motors and Settings pages read and write the API
3. Tests	TestRun, Test ID sequence, switch on, start, stop, assign, claim, channel, terminals, Edit details, vibration	All test actions work from the UI with the same rules
4. Telemetry	Ingest API, readings, minute trend, offline detection, WebSockets	A real device streams; Monitor (incl. Live readings), Device and Motor pages update live
5. Evaluation	Checks, settle rule, heating model, results per test, dashboard summary	Server conditions match the current UI on recorded data
6. Results, exports and audit	Results queries and rules, readings log, CSV / PDF exports for every download, activity with device tags, change log	Results and Activity pages run on the API; every download asks CSV / PDF and matches today's files
7. Hardening	Rate limits, backups, retention, deployment, load test (10 devices × 2 channels)	Ready for the shed

Acceptance tests to port

The front end has browser tests for every flow; re-run them against the API build — expected behaviour must not change:

- Sign-in, redirects (`?next=` , external links ignored), disabled accounts, sign-out.
- Devices: add / edit / delete with every validation message; offline device alert.
- Motors: register, duplicate ID / stator, edit, assign, delete; Switch on and Add details.
- Monitoring: stop (vibration asked, validation), start again (new Test ID, history kept), change channel, change terminals.
- Edit details and the test's change log.
- Users, roles (view implied by create / update / delete), settings validation, password change.
- Every download button asks CSV or PDF; file contents (no doubled sensor names; Limit check wording).
- Permissions for all four roles on every page.
- Layout at 320 px to 1920 px, landscape, light and dark.